GitHub 週間スター上位レポート

2026-10-30 作成 ／ 対象: 2026-10-23 以降に作成されたリポジトリ
テーマ: LLM、AI agent

#	区分	リポジトリ	★	一言で
1	テーマ	andyhuo520/digital-human-koubo	1,522	写真1枚と一文の指示から、AIエージェントが台本作成・リップシンク生成・編集・パッケージングまで自動で行い、数字人（デジタルヒューマン）の口播動画を作るAgent Skill
2	テーマ	DenisSergeevitch/ex-ussr-kinship-research	1,282	旧ソ連・ロシア帝国圏の家系調査を、証拠重視で進めさせるAIエージェント向けスキル（Codex、Claude Code対応）
3	テーマ	ccch1mneyyy/dsh-go	1,243	DeepSeek Harness（DSH）をAndroidスマホ上で直接動かし、PCやサーバー上のDSHもリモート操作できる非公式Flutterクライアント
4	テーマ	qybaihe/bla	1,088	AIリバースエンジニアリングエージェント（REA）からソフトウェアを守る防御専用シールドツール
5	テーマ	anonnerdfrenzy/survival-llm	817	インターネット不要で自分のPC上で動く、出典を必ず示すサバイバル・防災向けAIアシスタントとオフライン百科ライブラリ
6	総合	unclebob/ub-mode	5,052	AIコーディングエージェントに規律あるソフトウェア工学原則を守らせるためのスキル定義（SKILL.md）
7	総合	rddmwmddr/android-ad-cleaner	4,901	中国メーカー製Androidスマホのシステム広告・ホット検索・営業通知を、root不要かつ元に戻せる形でまとめてオフにするツール
8	総合	JustAResearcher/gpu-unlock-ada	4,545	RTX 4090 など Ada 世代 GeForce の FP32 累算テンソル演算を、NVIDIA ドライバ起動前に解放する Linux カーネルモジュール
9	総合	interplains/Zero-Breach-Toolkit	3,699	Pegasus などの傭兵型スパイウェアの痕跡を、PC・Android・iPhone バックアップからオフラインで調べる Python 製ツール
10	総合	anhao/ai-pose-gallery-zh	3,609	日本語のAI人体ポーズ参考画像サイトを全量ミラーし、中国語化した依存ゼロの静的ギャラリー

テーマ 1. andyhuo520/digital-human-koubo

https://github.com/andyhuo520/digital-human-koubo

★ 1,522 ／ フォーク 344 ／ 言語 TypeScript ／ 作成 2026-10-26 ／ MIT

該当テーマ: AI agent

写真1枚と一文の指示から、AIエージェントが台本作成・リップシンク生成・編集・パッケージングまで自動で行い、数字人（デジタルヒューマン）の口播動画を作るAgent Skill

概要

Claude Code や Codex などのAIエージェントに読み込ませる Agent Skill（SKILL.md 形式）とスクリプト群で、1枚の写真から固定の数字人を作り、口播（トーキングヘッド）動画を自動で仕上げます。映像生成は Renoise MCP 経由の Grok Video 1.5 などで行い、whisper による台本照合、音声の統一処理を経て、Remotion で編集・レンダリングします。トップクリエイター25本の動画をフレーム単位で分析して作った48種類のパッケージングコンポーネントと、5つの編集スタイルを備えている点が特徴です。クレジット消費前に必ず見積もりを提示し、ユーザーの承認を待つコストゲートも組み込まれています。

要点

- 写真1枚から persona.json で固定の数字人を定義し、毎回同じ人物で動画を作れる
- Renoise MCP 経由で Grok Video 1.5 を使い、15秒以下の区間ごとにリップシンク動画を生成
- whisper による台本照合とフレーム検査、音声統一処理で品質を担保
- 5つの編集スタイルと48種のパッケージング部品を Remotion で組み合わせて編集
- 生成前にクレジットを見積もり、承認を得てから実行するコストゲートを搭載

何をするものか

AIエージェントに「この数字人で30秒のライブ配信広告を作って」のように自然文で依頼すると、台本作成、リップシンクする数字人の映像生成、音声の整音、字幕、プロのYouTuber風のパッケージングまでを自動で行い、完成した MP4 を出力する Agent Skill です。README の例では、一文の指示だけから27秒の直播（ライブコマース）広告を生成し、コストは Renoise の482クレジットだったとされています。リポジトリには主スキルの digital-human-koubo と、スタイルA（金融・テック解説）用エンジンの finance-explainer の2つのスキルが含まれています。デモ動画に登場する人物はすべてAI数字人で、「内容由 AI 数字人演绎」のラベルが付けられています。

主な機能

写真から作った数字人の顔・衣装・声の説明・プロンプトテンプレートを persona.json に保存し、GPT Image でライブスタジオや教室、京都の街並みなど任意のシーンに合成してから話させます。台本は15秒以下の区間に分割して生成し、whisper で文字起こしして台本と照合、各区間4フレームを検査して、Grok が字幕を焼き込んでしまった区間は再生成します。音声は1/3オクターブのスペクトルマッチング、ディエッシング、共通のルームリバーブ、ラウドネス正規化で統一します。人物の背後に大きな文字を置く TextBehind 効果や、英語・日本語・韓国語・フランス語・スペイン語・アラビア語での多言語生成（whisper による照合で94〜100%）にも対応しています。

仕組み・パイプライン

処理は次の順に進みます。まず Renoise MCP・ffmpeg・whisper のセットアップを確認します。次にコンテンツを診断して5スタイルから2つを推薦し、一度の質問で brief.json を確定します。続いて script.json を作り（1区間15秒以下、中国語で約5.4字/秒）、クレジットを見積もって承認を得たら第1区間を生成し、確認後に残りを一括生成します。生成物は CDN 対策としてレンジ分割ダウンロードし、whisper とフレームで検査、音声統一処理を経て単語単位のタイムライン tl.json を作成します。最後に Remotion でスタイルテンプレートと既定のパッケージングを適用してレンダリングし、遷移中フレームとラウドネス（-14 / -16 LUFS）を確認します。パッケージング部品は pack リストに名前で指定し、タイミングは "3:积分" のような「区間:単語」形式で書きます。

技術構成

編集テンプレートは TypeScript と Remotion 4 で書かれており、huazi-916（9:16のデータ駆動型テンプレート）、packaging（48部品キットとギャラリー）、reference 配下の launch-169・tutorial-169・multihost-169 があります。補助スクリプトは Python とシェルで、build_tasks、rangedl、assemble、align、check_edit、cutout_video、voicefx/unify、doctor.sh、new_edit.sh、make_assets などが揃っています。人物の切り抜きには Swift（personmask.swift）で macOS の Vision を使います。音声認識は whisper.cpp、地図表示は d3-geo・topojson-client・world-atlas、アニメーション絵文字は Google Noto Emoji Animation を使っています。任意で Xiaomi MiMo TTS による声のデザインやクローンにも対応します。

使い方・導入方法

git clone したあと、Claude Code なら ~/.claude/skills/、Codex なら ~/.codex/skills/ に2つのスキルフォルダをコピーします。WorkBuddy や OpenClaw などでは zip にしてインポートします。次に renoise.ai でアカウントを作り、Renoise MCP を接続します。Claude Code では /plugin marketplace add ArcoCodes/renoise-plugins-official を、Codex では codex mcp add renoise --url https://renoise.ai/api/public/mcp を使い、ログインはブラウザで行うため API キーは不要です。ローカルには ffmpeg、Python 3（numpy・Pillow）、Node.js 18以上、whisper-cli と ggml-medium-q5_0.bin モデルを用意し、doctor.sh で不足を確認します。効果音とプレースホルダーのBGMは make_assets.py でコード生成できます。

注目ポイント・注意点

映像生成は Renoise の有料クレジットを消費します。目安として27秒の広告で482、約4分半のチュートリアルで約3,250クレジットです。TextBehind 効果は Apple Vision を使うため macOS 専用で、それ以外はクロスプラットフォームとされています。Grok は自前の音声から口の動きを作るため、後から別の声に差し替えると約100msのずれが出ます。リポジトリ内の material: ID は作者のアカウント用なので、自分で数字人を登録し直す必要があります。動作確認済みと明記されているのは Claude Code のみで、他のエージェントは「動くはず」という扱いです。Remotion は従業員4人以上の企業では商用ライセンスが必要です。また、本人または許可を得た顔・声のみを使うこと、AI生成である旨を表示することなど、責任ある利用のルールが明記されています。

技術構成: TypeScript、Remotion 4、React (TSX)、Python 3 (numpy, Pillow)、Shell、Swift (macOS Vision)、ffmpeg、whisper.cpp、Node.js 18+、Renoise MCP (Grok Video 1.5, GPT Image 2.5, Wan 3.0)、Xiaomi MiMo TTS（任意）、d3-geo / topojson-client / world-atlas、Google Noto Emoji Animation

向いている人: Claude Code や Codex などのAIエージェントを使って、短尺広告・商品紹介・解説・チュートリアルといった口播動画を量産したい個人クリエイターやマーケター、小規模チームに向いています。プロ級のパッケージング手法を Remotion のコンポーネントとして学びたいエンジニアや、Agent Skill と MCP を組み合わせたワークフロー設計の実例を知りたい開発者にも参考になります。Renoise の有料クレジットと、ローカルでコマンドを実行できる環境が前提です。

テーマ 2. DenisSergeevitch/ex-ussr-kinship-research

https://github.com/DenisSergeevitch/ex-ussr-kinship-research

★ 1,282 ／ フォーク 134 ／ 言語 Python ／ 作成 2026-10-25 ／ MIT

該当テーマ: AI agent

旧ソ連・ロシア帝国圏の家系調査を、証拠重視で進めさせるAIエージェント向けスキル（Codex、Claude Code対応）

概要

旧ソ連15か国とロシア帝国期を対象に、教会記録、民事登録、軍歴、強制移住・弾圧記録などからルーツを探す作業を、AIエージェントに慎重な家系調査員として行わせるAgent Skillです。発見した情報はすべて出典まで追跡できるようにし、記録そのものと解釈を分け、何が証明済みで何が未確認かをはっきり示すのが特徴です。主張の確からしさはパーセンテージではなく hypothesis や corroborated といった説明可能なステータスで管理し、監査スクリプトで裏付けの弱さを検出します。指示文はロシア語で書かれていますが、エージェントはユーザーの言語で応答します。

要点

- 旧ソ連15か国とロシア帝国期の家系調査に特化したAgent Skill
- 主張に hypothesis〜contradicted の説明可能な証拠ステータスを付与
- 同一記録の複製は independence_group で独立証拠とみなさない
- CSV台帳と audit_evidence.py で調査の再現性と整合性を検査
- 既定の成果物はHTMLレポート、GEDCOM出力やPDF変換にも対応

何をするものか

Codex や Claude Code などの Agent Skills ランタイムに組み込み、AIエージェントを旧ソ連圏・ロシア帝国圏専門の家系調査員として振る舞わせるためのスキルです。対象はロシア、ウクライナ、ベラルーシ、モルドバ、バルト三国、コーカサス三国、中央アジア五か国を含む旧ソ連15か国です。教会の出生・婚姻記録、民事登録、軍事記録、強制移住や弾圧のファイル、索引化されていないアーカイブのスキャン画像などを読み解き、出典をたどれる形で証拠を積み上げます。指示文はロシア語ですが、エージェントは英語やポーランド語、ドイツ語などユーザーの言語で応答し、史料は原綴りのまま引用して訳を添えます。

主な機能

調査の受付として、問い、既知の事実、存命者の有無、調べてはいけない範囲を整理します。人物、資料（artifact）、主張、関係、検索ログをCSV台帳で管理し、出来事の発生時点での州・郡・教区・登記所といった歴史的な管轄に当てはめて調べます。革命前ロシア語、ウクライナ語、ポーランド語、ルーマニア語、ドイツ語、イディッシュ語の綴りや父称、旧姓、翻字の変種一覧も作ります。同姓同名の人物は、家屋番号や両親、代父母、正確な生年月日、所属部隊などの区別できる証拠が出るまで別の person_id として扱います。さらにアーカイブや大学などへの問い合わせ文の作成と、許可がある場合の送信、CANOの記録請求テンプレート、FamilySearch の索引調査と MCP 連携、第一次世界大戦期の GWAR 記録の検索も備えています。

仕組み・アーキテクチャ

中心は SKILL.md で、作業の流れ、守るべきルール、言語方針がまとめられています。references フォルダには分野別の手引きがあり、情報源の優先順位、証拠プロトコル、地域ごとのガイド、プライバシー、ブラウザと CAPTCHA の扱いなどを扱います。地域ガイドにはベッサラビア、ガリツィア・ブコヴィナ、チェチェン人の強制移住、ドイツでの強制労働などが含まれます。情報源は原本を最優先とし、次に公式目録、索引、刊行史料集の順に当たり、家系樹やフォーラムは手がかり扱いにとどめます。assets にはCSV台帳や報告書のテンプレートがあり、scripts のPythonツールがケースの作成、資料のハッシュ化、証拠の監査を担います。監査では、引用された資料が実際にその independence_group に属している場合だけ独立証拠として数えるため、宣言だけで裏付けを作ることはできません。

技術構成

主要言語は Python で、多くのスクリプトは Python 3 の標準ライブラリだけで動きます。オプションの scan_montage.py は Pillow を、build_pdf.py は Chrome、Chromium、Edge といった Chromium 系ブラウザを必要とします。PDFのページをPNGに書き出す機能は pypdfium2 があるときに使えます。成果物はHTMLレポート（assets/report.template.html）が基本で、家系樹のエクスポートには GEDCOM 5.5.1（UTF-8）を既定とし、必要に応じて GEDCOM 7 も選べます。tests にはユニットテストが、evals にはスキルが起動する条件や推論を確かめるYAML形式のテストケースが置かれています。scripts/ca にロシアのルート証明書が同梱されていますが、その用途は README からは不明です。

使い方・導入方法

Node.js がある環境なら npx skills add https://github.com/DenisSergeevitch/ex-ussr-kinship-research で導入できます。-g を付けると、プロジェクトをまたいで使える個人用インストールになります。git clone で ~/.codex/skills/ や ~/.claude/skills/ に直接置く方法もあります。使うときは、例えば「1930年代にカザフスタンへ流刑された曽祖父の両親を教会記録から探して」のように、家族についての問いをエージェントに伝えます。ケースデータはスキルのフォルダの外に置き、scripts/init_case.py でケースを作成し、hash_artifact.py で資料の SHA-256 を記録し、報告書を書く前に audit_evidence.py で台帳を検査します。PDFは明示的に頼んだときだけ build_pdf.py で書き出されます。

注目ポイント・注意点

家系調査を「監視」に使わせない設計が明確です。存命者の現住所、電話番号、アカウントの調査は拒否します。CAPTCHA への挑戦は1回だけで、外部の解読サービス、ペイウォールや地域制限の回避、流出データベースの利用は行いません。文書やDNAデータを本人の同意なく第三者サービスへアップロードすることもなく、強制移住や養子縁組などのセンシティブな発見は中立的に報告します。init_case.py はスキルのフォルダ内にケースを作ることを拒否し、個人データの誤コミットを防ぎます。既存の家系樹は証明ではなく手がかりとして扱われます。一方で、家系樹ソフトのインストール、MCP の設定、外部の家系樹の自動同期はスキルの対象外です。アーカイブの請求様式は最新のものを各自で確認する必要があります。

技術構成: Python 3、Pillow（任意）、pypdfium2（任意）、Chromium系ブラウザ（PDF出力）、GEDCOM、CSV / YAML / HTML テンプレート、Skills CLI（Node.js、任意）、FamilySearch MCP（任意）
向いている人: 旧ソ連・ロシア帝国圏にルーツを持ち、先祖の教会記録や軍歴、強制移住・弾圧の記録をたどりたい人に向いています。Codex や Claude Code を使って、出典の追跡と証拠評価を伴う再現性のある調査をしたい家系

研究者や、アマチュアの系譜学者にも適しています。AIエージェント用スキルの設計例として、証拠管理や安全面のルールの作り方を学びたい開発者にも参考になります。

テーマ 3. ccch1mneyyy/dsh-go

https://github.com/ccch1mneyyy/dsh-go

★ 1,243 ／ フォーク 51 ／ 言語 Dart ／ 作成 2026-10-25 ／ AGPL-3.0
該当テーマ: LLM

DeepSeek Harness（DSH）をAndroidスマホ上で直接動かし、PCやサーバー上のDSHもリモート操作できる非公式Flutterクライアント

概要

dsh-goは、オープンソースのコーディングエージェント基盤DeepSeek HarnessのAndroid向けクライアント「DSH Go」です。アプリ内にUbuntu・Node.js・DSHの実行環境を構築でき、RootやTermuxなしでスマホ単体でエージェントを動かせるほか、SSHやDSH Remote経由でPC・サーバー上のDSHにも接続できます。ネイティブUIとWeb版の切り替え、アクセシビリティを使ったスマホ操作、端末間暗号化された会話共有、デスクトップに出てくるマスコット（Deepy・鯨娘）など、モバイルならではの機能を多数備えている点が特徴です。DeepSeek公式とは無関係のコミュニティ製プロジェクトで、AGPL-3.0で公開されています。

要点

- Root・Termux不要でスマホ内にLinux環境とDSHを構築して実行
- SSH/DSH Remote経由でPCやサーバー上のDSHも遠隔操作できる
- アクセシビリティ権限でAIに画面読み取り・タップ操作をさせる機能
- Flutter（Dart）UIとKotlinによるAndroidネイティブ実装の構成
- AGPL-3.0公開、sdk/とexamples/はMIT、名称・マスコットは対象外

何をするものか

DSH Goは、DeepSeek Harness（DSH）を手元のAndroid端末で扱うためのクライアントアプリです。動作モードは大きく二つあり、一つはアプリ内にUbuntu、Node.js、DSHランタイムをインストールしてスマホ自体でDSHエンジンを動かす「本機実行」、もう一つはSSHやDSH Remoteを通じてPCやサーバーで動いているDSHに接続する「リモート接続」です。スマホ上で動くのはDSHエンジンであり、モデルへのリクエスト自体はユーザーが設定したモデルプロバイダーに送られ、そのアカウントで課金されます。DeepSeek公式とは関係のない非公式クライアントである点はREADMEで明確に述べられています。

主な機能

チャット面では、モデル切り替え、画像送信、音声入力、ツール承認や質問への応答に対応し、生成途中での割り込み入力やキューに積んだメッセージの即時送信、完了したプロセスの1行への折りたたみができます。ネイティブUIと公式Web版をワンタップで切り替え可能です。そのほか、DSH Go向けのネイティブなツールカード・フォーム・パネルを提供するプラグインUI、回答の共有カード化と端末間暗号化された会話再生リンク、横画面・タブレット向けレイアウト、6種のテーマ色と壁紙連動、4種のアイコン、中国語と英語のUI切り替え、作業状況に応じて動くデスクトップペットなどがあります。

スマホ操作と安全設計

権限を許可すると、本機エンジンが画面の読み取り、タップ、入力を行う「手机操控」機能が使えます。確認レベルは3段階から選べ、支払い、パスワード、認証コード、システム設定に関わる操作は常にユーザーに戻す設計です。ファイル一覧にはSafetyGate、SensitiveWords、ProtectedPagesといったクラスがあり、こうした保護処理を担っていると見られます。通常モードではアクセシビリティ権限を使い、邪魔をしない「不打扰模式」はAndroid 14以上に加えShizukuまたは自己接続ADBが必要です。画面テキストやスクリーンショットがモデルに送られうる点には注意が必要です。

仕組み・アーキテクチャ

リポジトリはモノレポ構成です。app/にFlutterのUIとAndroid Kotlinのプラットフォーム実装があり、Kotlin側はruntime（ランタイムのインストール・プロセス管理・セーフモード）、phone（スマホ操作、McpServerを含む）、pet（オーバーレイ表示）、voice、notify、shareなど機能別にプラグイン化されています。packages/dsh_clientはDSH Web APIのDartクライアント、packages/dsh_remoteはDSH RemoteのDartクライアントです。runtime/ではLinuxランタイムのパッケージング、エンジンへのパッチ、アプリとのブリッジを扱います。ローカルエンジンは既定でPRoot互換起動で、開発者オプションで直接起動も選べます。さらにservices/、site/、infra/に共有サービスやWebサイト、デプロイ資料が含まれます。

使い方・導入方法

Android 8.0（API 26）以上が必要で、GitHub Releasesまたは公式サイトからAPKをダウンロードしてサイドロードします。一般的なスマホはarm64版、エミュレータやx86_64端末はx64版を使い、32bit ARM版はありません。初回起動時に本機へランタイムを入れるかリモートホストに接続するかを選び、モデルと認証情報を設定します。本機実行には約1GBの空きが必要です。ソースからビルドする場合はLinux x86_64上でtooling/setup-toolchain.shを実行し、source tooling/env.sh、bash runtime/build-runtime.sh arm64-v8a、DSH_BETA_TOKEN_FILE= bash tooling/build-apk.sh -f arm64 --keep-daemonの順に進めます。ツールチェーンはJDK 17、Flutter 3.47.7、Android SDK/NDKで、Dartは^3.13.5が要求されます。

注目ポイント・注意点

DSH Remote接続時は画像添付のみ対応、アプリ内でWeb版を開けない、プラグイン管理はオン/オフのみ、DS Botや会話ログのエクスポート非対応など、接続方式によって機能差があります。匿名利用統計の送信が既定でオンなので、気になる場合は初回ガイドか設定の「隐私与分享」でオフにできます。署名の異なる旧テスト版からは上書きインストールできず、アンインストールすると会話やLinux環境が消えます。ライセンスはAGPL-3.0ですが、DSH Goの名称・ロゴやマスコットはライセンス対象外です（ASSETS.md参照）。現状はAndroidのみで、iOSやHarmonyOSへの対応は予定として言及されています。Google Play配布向けの構成ではありません。

技術構成: Dart、Flutter、Kotlin、Android SDK/NDK、JDK 17、Gradle、PRoot、Ubuntu、Node.js、Shizuku、ADB、Bash、Python 3
向いている人: DeepSeek Harnessを使っていて、外出先でもスマホからエージェントを操作したい開発者や、PCを持ち歩かずにスマホ単体でコーディングエージェントを試したい人に向いています。FlutterとKotlinを組み合わせた

Androidアプリで、Linuxランタイムの同梱やアクセシビリティを使ったAI操作をどう実装しているかを学びたいエンジニア、DSH Go向けプラグインを作りたい人にも参考になります。

テーマ 4. qybaihe/bla

https://github.com/qybaihe/bla

★ 1,088 ／ フォーク 148 ／ 言語 TypeScript ／ 作成 2026-10-25 ／ MIT
該当テーマ: LLM、AI agent

AIリバースエンジニアリングエージェント（REA）からソフトウェアを守る防御専用シールドツール

概要

BLA（Blind LLM Agents）は、配布するバイナリやElectronアプリ、JSバンドル、Python、WebAssembly、SQLite、AIモデルなどを、REAのようなAIによるリバースエンジニアリング・エージェントから守るための防御ツールです。自作ソフトウェアを「ハードニング」することで、解析エージェントがリソース上限に達したり、おとり（デコイ）に誘導されてトークンを浪費したりする一方で、実行時の挙動は変わらないことを狙います。「Poison（汚染）→ Exhaust（枯渇）→ Hallucinate（幻覚）」という3段階が中心コンセプトで、MCPサーバーやコーディングエージェント連携として使えます。

要点

- REAなどAI解析エージェントへの対抗を目的とした防御専用ツール
- Poison・Exhaust・Hallucinateの3段階で解析を妨害
- 実行時の挙動・バイナリ整合性・フォーマット妥当性は保つのが鉄則
- 多数のフォーマット（ASAR/WASM/SQLite/モデル/EVM等）に対応
- MCPサーバーとして各種コーディングエージェントに統合可能

何をするものか

BLAは「Counter-REA」を標榜する防御ツールで、配布用のビルド成果物に対して監査（audit）→保護（protect）→検証（verify）のパイプラインを実行します。npx bla-agents harden . と打つだけで dist/ や build/、.asar、.wasm、.sqlite、モデルファイルなどを自動検出し、ワンコマンドで保護を適用します。狙いは、AIリバースエンジニアリングエージェントが成果物を開いたときに解析予算を超過させ、おとりを追わせ、トークンを浪費させることで、解析コストを引き上げる点にあります。READMEでは防御目的・自社ソフトウェア限定での利用が明記されています。

主な機能（3段階の防御）

BLAの防御はPoison・Exhaust・Hallucinateの3層で構成されます。Poisonはフォーマット的に妥当なカナリアやハニー・メタデータ、おとりの制御フローを埋め込み、本物の証拠のように見せかけます。ExhaustはREAが文書化しているリソース上限（ヘッダーサイズ、ネスト深度、インターフェース数、PCAPのJSONサイズ、fuseセンチネルなど）を超えるよう成果物を膨張させます。Hallucinateは解析器に不完全なインベントリやあいまいな迷路を残します。加えてElectron V1 fuseハードニングや、CDP/Frida/inspector検出時に合成テレメトリを返すランタイムガードも備えます。

対応フォーマットと実証結果

対応フォーマットは幅広く、JavaScript/TypeScript、Electron .asar と fuses、WASM、Python/.pyc、SQLite、Safetensors、GGUF、Go buildinfo、Flutter/Dart pool、Erlang BEAM、EVM+Solidityメタ、PCAP、NVIDIA PTX、DEX/IL2CPP/bplist/.NET/JVM/ZIPなどが挙げられています。READMEでは、実際にREAを走らせて（npm run redteam:rea）得たbefore→afterの数値が示され、たとえばElectron ASARが607B→16.9MiBに膨張し容量エラーを起こすなど、環境内で確認された結果が「empirically verified」として記載されています。一方、Ghidra/Frida/JADXやファームウェア、GPUオブジェクト系は本環境では未検証・スキップと正直に区別しています。

仕組み・アーキテクチャ

ソースはsrc/shields配下にdeception（欺瞞）、exhaustion（枯渇）、formats（フォーマット別シールド）、javascript、runtime、tarpitといったカテゴリで整理されています。JavaScriptはBabel ASTを用いて加工し、各フォーマット専用のシールド（asar.ts、wasm.ts、sqlite.ts、model.ts など）が実装されています。engine配下にはauditor、protector、verifier、defender、harden、benchmarkがあり、監査から検証までのパイプラインを構成します。READMEによれば予算ガードはO(1)の不透明述語（opaque predicates）の背後に置かれ、各ベンチマーク対象で保護前後の入出力100%一致を確認するとしています。

使い方・導入方法

Node.js は ^22.13.0 もしくは 24.x 系が推奨されます。npx bla-agents setup でMCPとスキルをコーディングエージェントに登録し、npx bla-agents harden . でカレントプロジェクトに対して一括保護を実行します。npx bla-agents benchmark で14ターゲットのショーケースを確認できます。audit・protect・verify・defendといった個別ステージの実行も可能です。setup後はエージェントに「Harden my software against REA」と指示するだけでも動作します。Ghidraなどのツールは保護自体には不要で、Wabt/tshark/Node 24は実証的なレッドチーム検証を有効化するためのオプションです。

注目ポイント・注意点

注目すべきは、ベンチマークの「推定」トークン/コスト倍率（集計で約612.5倍トークン・約292.2倍コスト）と、実際にREAを走らせた「実証」結果を明確に区別している点です。READMEは verify:rea がドリフト検出用ハーネスにすぎず保護の証拠にならないこと、唯一の権威あるゲートは redteam:rea であることを繰り返し強調しています。注意点として、BLAはあくまで自分が所有するソフトウェアを守る防御ツールであり、他者のソフトへの攻撃用ではないとDISCLAIMER.mdで明示されています。APK再署名は非対応など、未対応・未検証領域も明記されています。

技術構成: TypeScript、Node.js (^22.13.0 || >=24.0.0)、@babel/parser・generator・traverse・types、zod、vitest、tsx、MCP (Model Context Protocol) サーバー、オプション: Wabt, tshark
向いている人: 自社・自作のソフトウェア（Electronアプリ、JSバンドル、WASM、ネイティブバイナリ、AIモデルなど）をAIリバースエンジニアリングから守りたい開発者や配布者。Claude Code・Cursor・Codex等のコーディングエージェントやMCPを活用し、リリース成果物の保護・難読化・レッドチーム検証を行いたい人に向く。

テーマ 5. anonnerdfrenzy/survival-llm

https://github.com/anonnerdfrenzy/survival-llm

★ 817 ／ フォーク 68 ／ 言語 Python ／ 作成 2026-10-24 ／ NOASSERTION
該当テーマ: LLM

インターネット不要で自分のPC上で動く、出典を必ず示すサバイバル・防災向けAIアシスタントとオフライン百科ライブラリ

概要

Survival LLM は、停電時の食品の安全、やけどの手当て、水の浄化といった緊急時の疑問に、ネット接続なしで答えるデスクトップアプリです。医療・防災・ハウツー・百科事典など約150万記事のオフラインライブラリを Ollama 上のローカルLLMが検索し、回答に使った記事と該当箇所を必ず提示します。災害に備えたい一般の人を主な対象としつつ、モデルごとの正確性・安全性を独立AIジャッジで評価した結果や評価スクリプトも公開しています。さらに、モデルが実際には読んでいない出典を挙げた場合はアプリ側で検出し、実際に読んだ記事の一覧に置き換える仕組みを備えている点が特徴です。

要点

- 質問も資料もPC内で完結し、セットアップ後は完全オフラインで動作する
- すべての回答に出典記事と引用箇所を表示し、架空の出典はアプリ側で差し替える
- Wikipedia、WikiMed、CDC、FEMA、米軍マニュアルなど約150万記事を収録
- PCのメモリ量に応じてNemotron、Qwen、Graniteなどのモデルを自動で提案
- Ollama・pywebview・pypdf・truststore以外はPython標準ライブラリのみで構成

何をするものか

Survival LLM は、自分のコンピュータ上で動く無料のAIアシスタントと、サバイバル用のオフライン百科ライブラリをまとめたアプリです。停電・災害・応急処置といった場面での質問に対し、ローカルに保存された医療、緊急対応、ハウツー、百科事典の記事を検索し、その内容をもとに回答します。回答ごとに参照した記事が一覧表示され、クリックすると該当箇所がハイライトされた状態で、ライセンス情報とともに原文を確認できます。セットアップはインターネットがあるうちに行う必要がありますが、その後はネット接続なしで動作し、質問内容が外部に送られることはありません。ライブラリには Wikipedia のよく読まれる約93万記事のほか、WikiMed、MedlinePlus、WikEM、Ready.gov、CDC、FEMA や米軍のサバイバル・応急処置マニュアル、USDA の缶詰ガイド、wikiHow、iFixit、Appropedia などが含まれます。

主な機能

主な機能は次のとおりです。まず推論の可視化で、回答の下にある Show reasoning を開くと、モデルの思考過程や途中の検索、読んだ記事を確認できます。推論トークン量は Off、1,000、4,000、16,000、無制限から選べます。次にカスタマイズ機能で、Instructions ボタンからシステムプロンプトを編集でき、回答の長さ制限も変更できます。Your files には PDF、Word、EPUB、Webページ、テキストファイルを追加でき、発電機のマニュアルや家族の避難計画などもライブラリと一緒に検索対象になります。ただし、スキャン画像や写真は現時点では読み込めません。Memory 機能は、家族のアレルギーや所有している機材などを短いメモとして

保存し、以降のチャットで参照します。保存されたメモは一覧で編集でき、Undo やAIによる保存のオフにも対応しています。このほか、日次の更新チェックと自動アップデートの機能もあります。

仕組み・アーキテクチャ

質問はまずローカルモデル（Ollama）に渡され、モデルが search_library、read_article、show_article というツールを呼び出してライブラリを参照します。app/server.py はチャット画面を 127.0.0.1 でのみ配信し、各ターンの処理を core.py に任せます。core.py は回答のストリーミングとツール実行を担当します。library/search.py は index.sqlite に対して SQLite FTS5 による BM25 検索を行い、約1,500文字単位のパッセージを返します。その際、公式ソースは上位に来るよう調整されています。本文はプレーンテキストの Markdown シャードとして保存されています。memory.py は、人物に関する会話の後にモデルへ短い追加ターンを与え、JSON形式で記憶の追加・変更・削除を判断させます。library/myfiles.py はユーザーのファイル用に独立したインデックスを作ります。また、モデルが最後に書く Sources: 行を実際に開いた記事や検索結果と照合し、それ以外の出典が含まれていれば、実際に読んだ記事の一覧に置き換えます。

技術構成とモデル選択

実装は Python で、外部依存は Ollama、ウィンドウ表示用の pywebview、PDF読み取り用の pypdf、システムの証明書ストアを使う truststore に限られます。それ以外は標準ライブラリで書かれています。フロントエンドは app/static 配下の HTML、CSS、JS です。ライブラリの構築には libzim、lxml、trafilatura と、PDF処理用の pdftotext（poppler）を使います。モデルはメモリ量に応じて提案されます。128GB以上のMacには Nemotron 3.5 Super（80GB）、32〜96GBのMacには Qwen3.8 27B、32GB以上のWindowsには Nemotron 3.5 Lightning、16〜31GBのPCには Granite 4.2 8B が割り当てられます。16GB機向けの高速な選択肢として Gemma 4 E4B も用意されています。8GB機は、安全に推奨できるモデルがなかったため、ライブラリ検索のみの利用となります。

使い方・導入方法

Mac（Apple Silicon、macOS 14以降）では Survival-LLM-mac.zip を、Windows 10/11 の64bit版では Survival-LLM-windows.zip を展開して起動します。初回起動時は、未署名アプリとしての警告を許可する操作が必要です。起動後は画面の案内に従い、AIエンジン（Windowsでは Ollama 1.6GB をダウンロード）、AIモデル、ライブラリを選びます。ライブラリは Everything（8.8GB）と Essentials（1.3GB、Wikipedia と iFixit を除く）から選べます。所要時間は20分から2時間半程度です。ソースから動かす場合は python3 app/server.py を実行し、ブラウザで使うなら --browser を付けます。ターミナルからは python3 chat.py で質問できます。テストは python3 tests/run_all.py で実行でき、ライブラリの構築手順は library/fetch_zims.py と build_library.py で示されています。

注目ポイント・注意点

評価の透明性が際立っています。22問の停電・応急処置の質問について、正確性、安全性、懐疑的な再チェックという3者のAIジャッジによるブラインド評価を行い、結果と評価スクリプトを evals/ に公開しています。Nemotron 3.5 Super は正答率86%で有害回答0件でした。一方、Granite 4.2 3B は致命的になり得る誤答があったため、提供対象から外されています。ただし評価は Wikipedia 追加前の旧ライブラリで実施されたもので、配布版の品質を保証するものではないと明記されています。そのほかの注意点として、最大モデルは80GBあり、フルライブラリのインストールには一時的に約27GBの空き容量が必要です。また、自分用のデフォル

トビルドには再配布できない素材が含まれます。ライセンス表記は NOASSERTION ですが、README ではコードは MIT、ライブラリの内容は各ソースのライセンスに従うとされています。AIは誤りを含み得るため、表示される出典で内容を確認することが前提です。

技術構成: Python、Ollama、pywebview、pypdf、truststore、SQLite FTS5、libzim、lxml、trafilatura、poppler (pdftotext)、HTML/CSS/JavaScript、C (packaging/launcher.c)
向いている人: 停電や災害に備え、ネットが使えない状況でも医療・防災情報を引ける環境を手元に置きたい一般の人やプレッパー層に向いています。開発者にとっては、プライバシーを重視したローカルRAGアプリの実装例として参考になります。LLMの安全性・出典捏造の評価方法に関心がある研究者や学生にも、公開された評価スクリプトと結果が役立ちます。

総合 6. unclebob/ub-mode

https://github.com/unclebob/ub-mode

★ 5,052 ／ フォーク 281 ／ 言語 不明 ／ 作成 2026-10-25

AIコーディングエージェントに規律あるソフトウェア工学原則を守らせるためのスキル定義（SKILL.md）

概要

ub-mode は、AIエージェントがコードを書いたり変更したりする際に、TDD や根本原因の修正、小さなコミットといった規律ある開発原則を強制するためのエージェントスキルです。@poteto 氏のルール（poteto-mode / pstack）の一部を土台に内容を絞り込み、crapper・dryer・mutator といった決定的な品質ゲートツールと組み合わせている点が特徴です。「まず動かす」「次に正しくする」「実践」の3段階で原則が整理されています。Claude、Cursor、Grok などのエージェントでスキルを使い、AI生成コードの品質を一定に保ちたい開発者向けです。

要点

- AIエージェント向けのスキルで、SKILL.md を配置して /ub-mode で呼び出す
- Make it Work → Make it Right → Practice の3段階で原則を構成
- CRAP < 10、重複度 < 0.8 など数値基準の品質ゲートを採用
- crapper・dryer・mutator など同作者の補助ツールと連携する
- リポジトリは README.md と SKILL.md の2ファイルのみで構成

何をするものか

ub-mode は、AIコーディングエージェントにソフトウェア工学の規律を守らせるための「エージェントスキル」です。コードの作成や変更のたびに、テスト駆動開発、ドメインのモデリング、根本原因の修正、小さく順序立てたコミットといった原則に従うよう、エージェントの振る舞いを方向付けます。README によれば、@poteto 氏のルール（poteto-mode / pstack）の一部をもとに内容を絞り込み、決定的な品質ゲートツールと組み合わせたものです。ルールを文章で示すだけでなく、計測ツールの数値で品質を確認する前提になっている点が特徴です。

主な原則の構成

原則は3段階に分かれています。1つ目の First Make it Work には、Laziness Protocol、Foundational Thinking、TDD、Redesign From First Principles、Attack the Premise、Subtract Before You Add、Outcome-Oriented Execution、Build the Lever、そして Architecture（Model the Domain、Boundary Discipline、Type System Discipline、Make Operations Idempotent、Separate Before Serializing Shared State）が含まれます。2つ目の Then Make it Right には、Fix Root Causes、Sequence Work into Verifiable Units、Test Behavior (Not Implementation)、Crapper、Dryer、Mutator、Refactoring があります。3つ目の Practice は、Keep a Log、Guard the Context Window、Worktree Cleanup、Commit and Push です。

仕組み・連携ツール

ub-mode 本体は SKILL.md というスキル定義ファイルで、品質の計測は同作者の別リポジトリのツールが担います。unclebob/crapper は関数ごとに循環的複雑度、テストカバレッジ、CRAP スコアを算出し、すべての関数を CRAP < 10 に保ちます。unclebob/dryer は構造的なコード重複を検出し、類似度 0.8 以上の重複ブロックをなくします。unclebob/mutator はミューテーションテストを実行し、テストで覆われていないコードや生き残ったミュータントを洗い出します。unclebob/uml-viewer は、コンポーネント、カバレッジ、CRAP、ミューテーションスコア、依存方向を可視化します。各ツールの対応言語や詳細な使い方は README からは不明です。

使い方・導入方法

導入は、SKILL.md をエージェントのスキルディレクトリに ub-mode/SKILL.md としてコピーするだけです。配置先の例として、~/.grok/skills/ub-mode/SKILL.md、~/.claude/skills/ub-mode/SKILL.md、~/.cursor/skills/ub-mode/SKILL.md が挙げられています。配置後、エージェント上で /ub-mode を呼び出して使います。品質ゲートを実際に機能させるには、crapper、dryer、mutator などの補助ツールも別途用意する必要があると考えられますが、それらのインストール手順は README には書かれていません。

注目ポイント・注意点

『Clean Code』で知られる Uncle Bob（unclebob）のアカウントによるリポジトリで、スター数は5000を超えています。AIに任せると崩れがちなコード品質を、CRAP スコアや重複度、ミューテーションテストといった客観的な指標で縛るアプローチが注目点です。Guard the Context Window や Worktree Cleanup のように、エージェント運用ならではの原則が含まれている点も興味深いところです。一方で、ライセンスは明記されておらず、主要言語も不明です。各原則の具体的な内容は SKILL.md に書かれていると思われますが、今回の資料には本文が含まれていないため、詳細は README からは不明です。

技術構成: Markdown（SKILL.md によるエージェントスキル定義）、Claude / Cursor / Grok のエージェントスキル機構、unclebob/crapper、unclebob/dryer、unclebob/mutator、unclebob/uml-viewer
向いている人: Claude Code、Cursor、Grok などのAIコーディングエージェントを日常的に使い、生成コードの品質（テスト、複雑度、重複）を規律で担保したい開発者に向いています。TDD やリファクタリングといった Uncle Bob 流の開発原則を、エージェント運用にどう落とし込むかを学びたいエンジニアや学生にも参考になります。

総合 7. rddmwmddr/android-ad-cleaner

https://github.com/rddmwmddr/android-ad-cleaner

★ 4,901 ／ フォーク 440 ／ 言語 Python ／ 作成 2026-10-24 ／ MIT

中国メーカー製Androidスマホのシステム広告・ホット検索・営業通知を、root不要かつ元に戻せる形でまとめてオフにするツール

概要

OPPO/OnePlus（ColorOS）、小米・Redmi（HyperOS）、vivo（OriginOS）、Honor（MagicOS）など中国メーカー製Android端末には、システム広告やホット検索、ニュース推薦、マーケティング通知が多数組み込まれています。本リポジトリは、それらをadb経由でroot化・アンロック・ROM書き換えなしにオフにするためのツール群です。端末ごとの操作手順を「機種スクリプト」として記述し、uiautomator2で設定画面を自動操作する点が特徴です。AIエージェントに任せる方法、チャットAIに手順を教わる方法、手動チェックリスト、ノーコードのデスクトップクライアントという4通りの使い方が用意されています。

要点

- root・アンロック・ROM書き換え不要。システムとアプリが元々持つスイッチだけを操作する
- 通知は「内容与营销」などチャネル単位でオフにし、注文・配送・更新通知は残す
- 広告コンポーネントは pm suspend で停止。所有者の同意が必要で、コマンド1つで元に戻せる
- AGENTS.md を読ませて、AIエージェントに一括実行させる使い方を想定している
- Mac/Windows向けのGUIクライアントがあり、Pythonがなくても使える

何をするものか

中国メーカー製Androidのシステムに組み込まれた広告をオフにするツールです。対象は個人化広告、広告トラッキング（OAID）、プルダウン検索のホット検索ランキング、マイナス1画面のニュース、ブラウザの情報フィード、アプリストアのカルーセル・起動画面・フローティングキャンペーン、各アプリのマーケティング通知、クイックアプリ（インストール不要のミニプログラム）などです。一方で、検索、マイナス1画面のカード、ストアのインストールと更新、天気の警報、宅配通知といった実用的な機能は残す方針です。pm uninstall や pm disable-user によるアンインストール・無効化は行いません。systemパーティションの改変やモジュールの導入もしません。各アプリの利用規約を本人の代わりに同意することもなく、アカウント画面や電話番号画面のスクリーンショットも撮らないと明記されています。

主な機能と処理フロー

コマンドラインの入口は tool/run.py で、次のサブコマンドを順に実行します。devices で接続端末と対応する機種スクリプトの有無を確認し、check で読み取り専用の検査を行います。backup で変更前のスナップショットを取り、recipe で機種スクリプトを実行します。続いて chan-scan でマーケティング通知チャネルを一覧化し、chan-off でそれらをオフにします。quickapp はクイックアプリの検出と無効化、finish は後処理で、最後に dev-off --yes で開発者モードを解除します。復元方法は docs/05-還原.md にまとめられています。停止したパッケージと通知チャネルは、それぞれコマンド1つで元に戻せます。アプリ内のスイッチはチェックリストの手順に沿って手動で戻します。

仕組み・アーキテクチャ

中核となる tool/fast.py は、uiautomator2 を使って画面を読み取り、設定項目の入口を探してスイッチを切り替えるエンジンです。機種ごとの手順は recipes/<ブランド>-<型番>/recipe.py に書かれています。内容は「どのアプリを開き、どの文字をたどって設定ページに入り、どのスイッチを切って、どれを残すか」です。設定画面は型番やシステムのバージョンによって変わるため、スクリプトは型番とバージョンごとに管理されます。バージョンが一致しない場合は警告を出し、入口が見つからない手順はスキップして誤操作を避けます。ほかに、通知チャネル処理の chan_fast.py と channel_policy.py、接続管理の adb_link.py、待機処理の wait.py、新機種調査用の実験的な explore.py などがあります。アプリ名の取得には Java 製の applabels.dex が使われています。

技術構成

主要言語は Python で、requirements.txt で指定されている依存は uiautomator2==3.7.1 だけです。端末との接続は adb を前提としており、HDC を使う純正HarmonyOSには対応していません。デスクトップクライアントは gui/ 以下にあり、画面は pywebview、パッケージングは PyInstaller で作られています。Windowsインストーラーには installer.iss が使われているようです。Mac版は署名とAppleの公証を受けており、Apple Silicon版とIntel版が配布されています。tests/ には実機をつながずに動かせるエンジンの回帰テストが多数あります。GitHub Actions にはテストとWindowsビルドのワークフローが用意されています。

使い方・導入方法

使い方は4通りです。1つ目はAIエージェントに任せる方法で、PCとケーブル、コマンドを実行できるAIツール（Claude Code、Codex、Cursor、Trae、WorkBuddy など）を使います。READMEにある開始用の文面をコピーして渡すと、AIがリポジトリをクローンし、AGENTS.md を読んだうえで作業します。2つ目はスマホだけの場合で、DeepSeek、豆包、ChatGPT などのチャットAIに手順を案内してもらいます。3つ目はブランド別のチェックリストを見ながら手動でオフにする方法です。4つ目はデスクトップクライアントで、ケーブルを挿してボタンを押すだけで済み、Pythonは不要です。コマンドラインで使う場合は、pip3 install -r requirements.txt のあと python3 tool/run.py を実行します。事前にスマホ側で開発者モードとUSBデバッグを有効にしておく必要があります。

注目ポイント・注意点

対応機種表には、Honor、OnePlus、OPPO、小米、Redmi、vivo/iQOO など20機種以上が並んでいます。ただし状態は「実測で全工程通過」「経路のみ実測」「未対応」と機種ごとに異なります。同じブランドのスクリプトがあっても、その機種専用の対応とは限らないと明記されています。表にない機種は docs/04 に沿って自分でスクリプトを書き、PRを送ることが推奨されています。旧バージョンのクライアントは自動更新されないため、新しい版を手動でダウンロードし直す必要があります。利用前のバックアップが推奨されており、使用によって生じた問題は利用者の自己責任とされています。アプリの起動時広告（スプラッシュ広告）は .out-of-scope に置かれており、対象外と考えられます。

技術構成: Python、uiautomator2、adb、pywebview、PyInstaller、Java（applabels.dex）、GitHub Actions
向いている人: 中国メーカー製Android（ColorOS、HyperOS、OriginOS、MagicOS）のシステム広告や営業通知に悩んでいて、root化せずに安全かつ元に戻せる形で減らしたい人に向いています。コマンド操作に慣れていない人は

デスクトップクライアントやチャットAIを、開発者はCLIやAIエージェントを使えます。新機種のスクリプトを書いて貢献したい人や、UI自動化・adbの実践例を学びたい人の参考にもなります。

総合 8. JustAResearcher/gpu-unlock-ada

https://github.com/JustAResearcher/gpu-unlock-ada

★ 4,545 ／ フォーク 580 ／ 言語 C ／ 作成 2026-10-23 ／ GPL-2.0

RTX 4090 など Ada 世代 GeForce の FP32 累算テンソル演算を、NVIDIA ドライバ起動前に解放する Linux カーネルモジュール

概要

NVIDIA ドライバが GPU を初期化する前に SM の speed-select 設定を書き換え、RTX 4090 で FP16/BF16(FP32 累算)のテンソル命令スループットを約 2 倍に引き上げる Linux カーネルモジュールです。Zagitta/gpu-unlock の Linux モジュールとパッケージングに、TOTORONG/AD103_win_unlocker で公開された AD102 向けペイロードを統合したフォークです。NVIDIA のファームウェアは配布せず、ビルド時に各自のマシンで公式入力を取得・ハッシュ検証して生成します。改変していない純正 NVIDIA ドライバのまま動作する点と、カードごとに詳細な実測エビデンスを公開している点が特徴です。

要点

- RTX 4090 で FP16/BF16→FP32 累算が約165→331 TF/s に向上
- INT8・dp4a・スカラー FP32 は変化なし、実アプリの高速化は限定的
- 純正 NVIDIA ドライバのまま、initramfs で先に動作する方式
- 4070 Ti SUPER など多数の Ada GPU は個別フラグ付きの実験的対応
- ファームウェアはローカル生成で SHA-256 検証、GPL-2.0-only

何をするものか

Ada 世代の GeForce GPU では、FP16 や BF16 の入力を FP32 で累算するテンソル演算が、ハードウェア上の上限より低い速度に設定されているとされます。このモジュールは、NVIDIA ドライバがカードを掴む前に署名済みファームウェアとペイロードを実行し、SS0 に 0x88888888、SS1 に 0x8 を書き込んで speed-select を上書きします。その後 PCIe の Function Level Reset(FLR)を行い、未改変の NVIDIA ドライバに引き渡します。RTX 4090 では命令レベルで約 2 倍(例 160.4→320.4 TF/s)が報告され、TF32 や FP8 も命令発行レートで約 2 倍になる測定があります。

対応 GPU と実験的ターゲット

RTX 4090(10de:2684, AD102)は自動で AD102 用ファームウェアとペイロードが選ばれます。上流から引き継いだ GA102/GA104/GA103/GA106 系の Ampere 対応も残っており、RTX 3050 6GB(GA107)の成功報告もあります。RTX 4090 D、4080 / 4080 SUPER、4070 系、4060 系、ノート PC 向け 4090/4060/4050 などは実験的扱いで、ID ごとに rtx4090d_experimental や ad103_experimental などのフラグが必要です。多くは正確な PCI アドレス指定も必須で、targets=all では選ばれません。検証は 1 枚〜数枚程度で、特にノート PC は機種ごとに挙動が異なると明記されています。

仕組み・アーキテクチャ

パッケージはモジュールとファームウェアを initramfs に組み込み、mkinitcpio、initramfs-tools、dracut 向けの設定で gpu_unlock を nvidia より先に読み込ませます。カーネルは読み込んだ各アセットを固定の SHA-256 と照合してから DMA を開始し、ペイロードの完了マーカーと権限を確認、FLR 前後で speed-select の読み出し値を検証します。WPR2 が残った初期化済みカードや既にバインド済みのカードはスキップします。ペイロードは公開されたデバッグ用 booter の解析から導いた命令アドレスを使い、製品版 booter は GPU 内で認証・復号されます。

技術構成

本体は C で書かれた Linux カーネルモジュール(kernel/gpu_unlock)で、DKMS でビルドされ CONFIG_CRYPTO_LIB_SHA256 が必要です。Arch(makepkg)、Debian(dpkg-buildpackage)、RPM(rpmbuild)の各パッケージングを Makefile から扱えます。ビルド時に NVIDIA 610.43.03 のソースとバイナリを HTTPS で取得し、ハッシュ検証してファームウェアを生成します。診断用に読み取り専用の BAR0 サンプラ diagnostics/peek(C)と、CUDA の命令ベンチマーク・GEMM 検証 diagnostics/bench(CUDA、Python)が付属します。make test はモックによるホスト側テストです。

使い方・導入方法

git clone 後に make install を実行すると、ディストリビューションを判別してパッケージをビルド・インストールします。明示的に make install-arch、make install-deb、make install-rpm も使えます。カーネルヘッダと DKMS が必要です。導入後はカーネルコマンドラインに gpu_unlock.targets=0000:03:00.0 のように lspci -D で得た PCI アドレスを指定して有効化し、実験的ターゲットには対応フラグも付けます。初回は必ず電源を完全に切ってからのコールドブートが求められます。起動後は journalctl -k -b | grep gpu_unlock で「gpu_unlock: unlocked」と各チェックの結果を確認します。

注目ポイント・注意点

効果はテンソル命令の発行レートに関するもので、実際の cuBLAS GEMM では FP16 で 12〜34% 程度、形状によってはほぼ変化なしでした。量子化 MoE 推論では改善なしの報告もあり、電力上限下では消費電力が増えて頭打ちになる点も記されています。結果の多くは 1 枚のカード・少数回の起動に基づき、長期安定性は十分検証されていません。NVIDIA 起動後の手動ロードでは効果がなく、ファームウェアが停止しなかった場合は強制アンロードせず電源断が必要です。保証や規約面への影響は README からは不明なため、自己責任で慎重に扱うべきツールです。

技術構成: C、Linux カーネルモジュール、DKMS、CUDA、Python、Makefile、mkinitcpio / initramfs-tools / dracut、Debian パッケージング、RPM パッケージング、NVIDIA 610.43.03 ファームウェア(ローカル生成)
向いている人: RTX 4090 などの Ada 世代 GeForce を Linux で使い、FP16/BF16/TF32 の FP32 累算テンソル性能を検証したいハードウェア研究者や上級ユーザー向けです。カーネルモジュールや initramfs、ブートローダ設定を自分で扱え、検証不足のリスクを理解したうえで自分の GPU で実験できる人に向いています。一般的な AI 推論の高速化を手軽に求める用途には向きません。

総合 9. interplains/Zero-Breach-Toolkit

https://github.com/interplains/Zero-Breach-Toolkit

★ 3,699 ／ フォーク 393 ／ 言語 Python ／ 作成 2026-10-25 ／ Apache-2.0

Pegasus などの傭兵型スパイウェアの痕跡を、PC・Android・iPhone バックアップからオフラインで調べる Python 製ツール

概要

Zero Breach Toolkit は、Pegasus、Predator、Graphite、Candiru など既知の傭兵型（mercenary）スパイウェアの痕跡を、Windows・macOS・Linux のコンピューターと、Android の収集データや iPhone のローカルバックアップから探す無料のオープンソースツールです。スキャンは読み取り専用で、スキャン中はネットワーク通信を行いません。どのレポートにも、何が見つかったか、証拠の強さ、何を確認できなかったかが明記され、「検出なし」を安全の証明として扱わない設計になっています。技術知識のないジャーナリストや活動家でも使えるよう、手順ガイドと対話メニューが用意されている点も特徴です。

要点

- Amnesty、Citizen Lab、MVT など29ソース・約12,400件の公開インジケーターと照合する
- スキャン中は Python のネットワーク呼び出しをブロックし、読み取り専用で動く
- PC のブラウザ履歴や各種ログ、Android（AndroidQF）、iPhone バックアップに対応
- 証拠の強さの評価と「確認できなかった範囲（gap）」を必ず報告する
- 標準ライブラリのみで動き、配布用単一ファイルは決定的ビルドで再現検証できる

何をするものか

国家などが使う傭兵型スパイウェアについて、研究機関が公開した既知のインジケーター（ドメイン、ファイルハッシュ、アプリ名、証明書など）が、端末の記録に残っていないかを調べるトリアージツールです。対象は Windows・macOS・Linux のコンピューターに加え、AndroidQF で収集した Android データ、Finder や Apple Devices で作った iPhone のローカルバックアップ（暗号化の有無を問わない）、展開済みファイルシステム、sysdiagnose アーカイブです。README は、未公開の新しいスパイウェアは検出できないこと、専門家によるフォレンジック調査の代わりにはならないことをはっきり書いています。今まさに標的にされていると感じる人には、ツールを使う前に Access Now Digital Security Helpline や Amnesty International の Security Lab に相談するよう勧めています。

主な機能と検査対象

Windows では、Chrome・Edge・Firefox・Tor Browser など多数のブラウザの履歴、Cookie、HSTS、キャッシュを調べます。ほかに Mark of the Web、Win+R の入力履歴、Microsoft Defender の検出履歴と除外設定、PowerShell ログ、Shim cache、Prefetch、Amcache、DNS キャッシュ、hosts ファイル、スタートアップ登録なども対象で、管理者権限は不要です。macOS と Linux では Safari の各種データベース、LaunchAgent・systemd・cron などの自動起動設定、シェル履歴、システムログを確認します。Android ではサイドロードされたアプリの権限、インストール履歴、SMS 内のリンク、ブートローダーのアンロック状態などを見ます。

iPhone では SMS・iMessage・WhatsApp、Safari、クラッシュレポート、構成プロファイルなどを読みます。走査範囲は --scan-type で default、full、targeted、browser から選べます。

仕組み・アーキテクチャ

コードは scan/ 以下にあります。エントリーポイントは scan/zero-breach-toolkit.py で、本体パッケージの scan/zbtriage/ がインジケーターの読み込み、ネットワーク遮断（netguard.py）、レポート生成を担います。modules/ 配下には browser、host、posix、android、ios、hashscan、execution など、検査ごとのモジュールが置かれています。リーダーは共通のレジストリで管理され、プラットフォーム選択、カバレッジ説明、単一ファイルビルドに使われます。SQLite の WAL レコードは一時コピー上で検証してから再生し、ソース DB は変更しません。JSON の検出結果にはソースのハッシュとレコードの位置情報が残ります。Candiru 向けの YARA ルール（third_party/candiru）も同梱されています。tests/ には iOS 15〜18 のテーブル構造を含む大規模なテストスイートがあります。

技術構成

Python 3.8 以上で動き、追加パッケージなしで使えます。暗号化された iPhone バックアップを扱うときだけ、任意で cryptography パッケージが必要です。これは requirements-backups.txt からバージョンとハッシュを固定してインストールする方式で、自動インストールはされません。Android の収集には外部ツールの AndroidQF を使います。実験的な --collect-android は AndroidQF v1.8.3 を、SHA-256 を固定した状態でダウンロードして実行します。build/ にはリリース用の単一ファイルを作る bundle.py と、STIX、MISP、dnsmasq、hosts 形式でインジケーターを出力するスクリプトがあり、dist/ に生成物が置かれています。GitHub Actions でテストとリリース検証を行っています。

使い方・導入方法

GitHub から ZIP をダウンロードするか git clone し、ツールのフォルダで python -X utf8 scan/zero-breach-toolkit.py --update を実行して最新のインジケーターを取得します。インターネットを使うのはこの手順だけです。続いて python -X utf8 scan/zero-breach-toolkit.py を実行するとメニューが開き、Enter で標準チェックが始まります。所要時間はおよそ15分です。Windows では scan/run-scan.cmd をダブルクリックしても起動できます。macOS では事前にターミナルへフルディスクアクセスを許可する必要があります。主なオプションは、--quick（ハッシュ検査を省略）、--phone でスマートフォンのデータを指定、--json で機械可読レポートを出力、--share-json で共有用の最小サマリーを出力、--evidence-bundle で証拠 ZIP を作成、などです。終了コード 0〜4 で結果をスクリプトから判定できます。

注目ポイント・注意点

作者が匿名であることを前提に、自分で検証できる仕組みを重視しています。リリースの単一ファイルは決定的ビルドで、タグから再ビルドすれば SHA-256 を照合できます。GPG 署名も付いています。一方、署名済みの Windows 実行ファイルはなく、今後も予定されていません。--describe や scan/CONTRACT.md、skills/ 配下の SKILL.md など、AI アシスタントから使うことを想定した仕組みがあるのも珍しい点です。注意点もいくつかあります。完全なレポートにはユーザー名、閲覧履歴、SMS の抜粋などの個人情報が含まれます。証拠バンドルは暗号化されません。ネットワーク遮断は OS レベルのサンドボックスではありません。「一致あり」は感染確定ではなく、詳しく調べるべきだという合図として扱われます。

技術構成: Python 3.8+、SQLite（WAL 処理）、cryptography（任意、暗号化 iPhone バックアップ用）、AndroidQF、YARA ルール（Candiru）、STIX / MISP、GitHub Actions、GPG
向いている人: スパイウェアの標的になりうるジャーナリスト、活動家、人権擁護者、市民社会の関係者と、その支援者に向いています。専門知識がない人でも手順ガイドに沿って使えます。デジタルセキュリティやフォレンジックの初動トリアージ、IOC ベース検出の実装を学びたいエンジニアや学生、オフラインで動く検査ツールを AI アシスタントやスクリプトに組み込みたい人の参考にもなります。

総合 10. anhao/ai-pose-gallery-zh

https://github.com/anhao/ai-pose-gallery-zh

★ 3,609 ／ フォーク 934 ／ 言語 HTML ／ 作成 2026-10-27 ／ NOASSERTION

日本語のAI人体ポーズ参考画像サイトを全量ミラーし、中国語化した依存ゼロの静的ギャラリー

概要

原作者ギア氏が公開していたAIポーズ参考図鑑は、メタデータがすべて日本語で、素材の一括ダウンロードにも対応していませんでした。本リポジトリはその1439件分の画像を収録しています。サムネイルと原寸画像で計2878ファイルです。あわせてメタデータを中国語に全訳し、依存ライブラリなしで動くレスポンシブなギャラリーを用意しています。Cloudflare Pagesへそのままデプロイでき、Claude Codeなどで使えるAgent用のSkillsも同梱されており、AI画像・動画生成のポーズ参考として使える点が特徴です。

要点

- 288ポーズ・1439件・2878ファイル（約659.5MB）を収録
- 日本語メタデータを全項目中国語化し、323件の対訳表も付属
- index.html単体で動く依存ゼロのギャラリー
- Cloudflare Pages向けのビルドスクリプトとキャッシュ設定を同梱
- AI Agent向けSKILL.md形式のポーズ検索スキルを5種提供

何をするものか

AI絵画、3D、撮影の参考になる人体ポーズ画像集の中国語版ミラーです。元サイトは ai-pose-gallery.pages.dev で、原作者はギア（@giaaibusi）、中国語版はAnhaoがメンテナンスしています。作業は3つです。1つ目は、1439件分のサムネイル（JPG 300×400）と原寸画像（PNG 900×1200）を全件ローカルに保存したこと。2つ目は、4つの体系、26の分類、288のポーズ名、5つの視点を漏れなく中国語訳したこと。3つ目は、元サイトの機能を再現したフロントエンドを用意したことです。体系の内訳は、全身単人・標準系365件、全身単人・セクシー系399件、上半身単人400件、二人ポーズ275件です。なお、GitHubの説明文にある「1439组 / 2878张」は、READMEの記述（288組・1439件・2878ファイル）と数え方が異なるので注意してください。

主な機能

ギャラリーは4体系のタブ切り替えに対応し、各タブに画像数とポーズ組数が表示されます。ポーズ名、番号、ID、分類キーワードで検索でき、スラッシュキーで検索欄にすぐ移動できます。動作分類と5種類の視点（正面、斜め、側面、俯瞰、煽り）を組み合わせて絞り込めます。表示はポーズ単位のグループ表示と1枚ずつの表示を切り替えられます。ライトボックスではまずサムネイルを表示し、その後に原寸画像を読み込みます。PNGの直接ダウンロード、IDのコピー、新規ウィンドウでの表示にも対応しています。キーボードでは左右キーで角度、上下キーでポーズを切り替え、Escで閉じます。スマートフォンではスワイプ操作と2列表示に対応し、絞り込みパネルを折りたためます。

仕組み・アーキテクチャ

index.html 1枚のシングルページアプリで、fetchを使って data/pose-zh.json を読み込み描画します。pose-zh.json には、id、体系、姿勢編号、分类、姿势名称、角度、缩略图、原图、そして検索用の結合文字列「检索」が含まれ、これを使ってあいまい検索を行います。元の日本語データ pose.json と meta.json は改変せずに残されており、比較や再取り込みに使えます。データ生成は scripts/ 以下のPythonスクリプトで行います。i18n.py は重複を除いた323件の日中辞書、apply_translation.py は訳文を当てはめて中国語データを生成するスクリプトです。download.py は48スレッドの並列ダウンロード、途中再開、自動リトライに対応し、verify.py はファイルの存在、サイズ、画像ヘッダーを検証します。SEO対策として、canonical、Open Graph、Twitter Card、JSON-LD（WebSite と ImageGallery）、robots.txt、sitemap.xml も組み込まれています。

使い方・導入方法

JSONをfetchで読み込むため、index.html をダブルクリックで開くとCORSにより読み込みが止められます。ローカルでは python3 -m http.server 8000 を実行し、http://127.0.0.1:8000 を開いてください。画像の再取得は python3 scripts/download.py で行います。並列数は WORKERS=64 のように環境変数で変えられます。訳文を修正した場合は apply_translation.py を実行し、検証には verify.py を使います。Cloudflare Pagesにデプロイする場合は、Framework preset を None、Build command を bash scripts/build.sh、出力先を dist に設定します。コマンドラインからは npx wrangler pages deploy dist で直接アップロードできます。Forkして別ドメインで公開する場合は、index.html、robots.txt、sitemap.xml 内の絶対URLを書き換える必要があります。

Agent Skills

skills/ 以下に、SKILL.md形式のAgentスキルが5つあります。standard、upper、sexy、couple、all の5種類で、AI Agentがポーズ名と視点から白モデル画像を検索し、画像・動画生成の参考にできます。各スキルは references/catalog.csv を持ち、画像パスは ../../img/<id>.png という相対パスでリポジトリ内の img/ を直接参照します。そのため、リポジトリ全体をクローンしたまま、スキルディレクトリを ~/.claude/skills/ などへコピーではなくシンボリックリンクで配置する必要があります。リポジトリを移動したときはリンクを張り直してください。画像を内蔵した単独コピー可能な版としては、別リポジトリの jasoncai0/pose-reference-skills が紹介されています。

注目ポイント・注意点

画像の一部は成人向けのポーズ参考で、セクシー系と二人ポーズのスキルは18+と明記されています。用途は芸術創作、動作参考、フロントエンド学習に限られ、未成年の閲覧は不可とされています。ライセンスにも注意が必要です。MITライセンスが適用されるのは scripts/ と index.html のコードだけで、img/、thumbs/、data/ の画像と元データの著作権は原作者ギア氏にあります。本リポジトリは画像に対していかなる権利も主張しておらず、原作者から要請があれば削除するとしています。容量は原寸画像だけで約637MBと大きいです。Cloudflare Pagesで出力先をルートディレクトリにすると、.git が原因でビルドに失敗する点もREADMEで注意されています。

技術構成: HTML、JavaScript（依存ライブラリなし）、Python（ダウンロード・翻訳・検証スクリプト）、Bash（build.sh）、JSON / CSV、Cloudflare Pages、wrangler、JSON-LD / Open Graph
向いている人: AI画像・動画生成や3D、イラスト、撮影でポーズ参考を探す中国語圏のクリエイターに向いています。依存ゼロの静的ギャラリー、Cloudflare Pagesへのデプロイ、SEO設定、Python製スクレイピングと翻訳パイプラインの実例を学びたいフロントエンド学習者にも参考になります。Claude CodeなどのAgentにポーズ参照スキルを組み込みたい人にも使えます。ただし成人向けコンテンツを含み、画像の権利は原作者にあるため、利用目的には注意が必要です。